

WIPRO EMBEDDED TRAINING PROGRAM

Individual Capstone Project • 6-Stage SDLC Technical Report

VDEVPULSE

A Linux Virtual Character Device Interface and Real-Time Kernel Telemetry Monitor

Candidate Name: Priyansu Das

Training Track: Embedded Systems & System Software

Language & Standard: C++17 / POSIX API

Operating System: Linux (Kernel ≥ 4.15 / Ubuntu / WSL2)

Build & Test Suite: CMake 3.14+ / CTest (100% Pass Rate)

Source Repository: <https://github.com/priyansudas07/Wipro-Embedded-Training>

Submitted in partial fulfillment of the Wipro Embedded Training curriculum requirements.

Date of Submission: September 2026

1. PROJECT OVERVIEW & OBJECTIVES

1.1 Project Description

VDevPulse is an embedded system software daemon written in modern C++17 that creates a virtual Linux character device interface (at `/tmp/vdevpulse`) and continuously samples CPU, RAM, system load averages, and uptime directly from Linux kernel synthetic filesystems (`/proc/stat`, `/proc/meminfo`, `/proc/loadavg`, `/proc/uptime`). It provides applications with standard POSIX device file operations (`open`, `read`, `write`, `close`), virtual IOCTL command handling, an automated threshold alert engine, interactive text query protocol, in-memory telemetry history buffer, and structured JSON export streaming.

1.2 Why I Built This Project (Engineering Rationale)

During embedded systems training, I observed that monitoring hardware health on constrained embedded Linux hardware (such as Raspberry Pi, BeagleBone, and industrial edge controllers) is often inefficient because existing monitoring software is either too heavy or requires extensive external dependencies.

I designed and built VDevPulse with specific engineering objectives:

- **Low-Overhead Embedded Monitoring:** Most modern telemetry daemons rely on Python runtimes, JVMs, or heavy Go agents consuming 50–200 MB of RAM. VDevPulse operates with a memory footprint under 15 MB RSS and near-zero CPU overhead.
- **UNIX Device Abstraction ("Everything is a File"):** By exposing telemetry through a named FIFO device node (`/tmp/vdevpulse`), any user application or shell script can inspect hardware health using standard POSIX file stream operations (`cat`, `read()`) without needing proprietary client libraries or network sockets.
- **Practical Linux System Programming Mastery:** The project demonstrates practical implementation of core POSIX concepts: non-blocking FIFO stream management (`mkfifo`, `O_RDWR | O_NONBLOCK`), virtual IOCTL command dispatch, kernel procfs parsing, POSIX signal handling (`SIGINT` / `SIGTERM`), and thread-safe C++17 synchronization (`std::mutex`, RAII lock guards).
- **Safe User-Space Emulation:** Developing native kernel modules (`.ko`) requires root compilation against exact kernel headers and risks fatal kernel panics during bugs. VDevPulse achieves character device semantics entirely in user-space, making it safe, crash-isolated, and 100% portable across Linux distributions.

1.3 Practical Utility & Real-World Applications

1. **Industrial Edge Gateway Health Monitoring:** Continuously tracks CPU utilization, RAM pressure, and system load averages on edge nodes, detecting runaway processes before they trigger hardware watchdog reboots.
2. **Interactive Device Query Protocol:** External processes can query live metrics directly over the device node using commands like `GET_CPU`, `GET_MEM`, `GET_LOAD`, `GET_JSON`, and `GET_HEALTH`.
3. **Automated Threshold Alert Engine:** Evaluates live metrics against thresholds in `vdev_policy.json` (e.g. CPU > 85%, RAM > 90%) and dynamically updates health status with warning alerts.
4. **Structured JSON Export:** Directly stream metrics as JSON objects for integration with web dashboards or analytics tools.
5. **Historical Telemetry Ring Buffer:** In-memory circular buffer preserving recent samples for trend inspection via `vdevpulse history`.

1.4 Problem Statement & Comparison

Tool / Approach	Operational Characteristic	Key Limitation in Embedded Systems	VDevPulse Advantage
top / htop	Interactive terminal UI	Cannot be programmatically queried as a standard character device stream.	Standard named FIFO character device node (<code>/tmp/vdevpulse</code>).
Python / Go daemons	Background scripts	High memory footprint (50–200 MB RSS), large runtime dependencies.	Under 15 MB RSS, zero external dependencies, pure C++17.
Prometheus Agents	HTTP/TCP metrics exporter	Requires network stack overhead, open ports, and scrape setup.	Self-contained local daemon reading kernel procfs directly (< 1 ms latency).
Kernel Modules (.ko)	In-kernel C drivers	Requires root compilation; bugs cause fatal kernel panics.	User-space character device emulation; completely safe and crash-isolated.

2. REQUIREMENTS & ARCHITECTURE

2.1 Core Subsystems & Features

Subsystem	Source Files	Key Features & Responsibilities
Telemetry Engine	telemetry_monitor.cpp	Parses <code>/proc/stat</code> (CPU), <code>/proc/meminfo</code> (RAM), <code>/proc/loadavg</code> (Load/Threads), <code>/proc/uptime</code> ; evaluates threshold rules; formats JSON; maintains 60-sample history buffer.
Device Manager	device_manager.cpp	Manages <code>/tmp/vdevpulse</code> FIFO node; non-blocking read/write; processes interactive queries (<code>GET_CPU</code> , etc.); executes IOCTLs (<code>START</code> , <code>STOP</code> , <code>RESET</code> , <code>GET_STATS</code> , <code>SET_RATE</code>).
Policy Engine	config.cpp	Loads sampling rate, threshold limits, and output format from <code>vdev_policy.json</code> with safe default fallback.
Logger	logger.cpp	Singleton thread-safe synchronized logging with ANSI color coding and file persistence.

2.2 Core Data Structures

```
// include/vdevpulse/telemetry_monitor.hpp
struct SystemTelemetry {
    double cpu_usage_pct      = 0.0;
    long   memory_total_mb    = 0;
    long   memory_used_mb     = 0;
    long   memory_free_mb     = 0;
    double memory_usage_pct   = 0.0;
    long   uptime_seconds     = 0;
    double load_1m            = 0.0;
    double load_5m            = 0.0;
    double load_15m           = 0.0;
    int     running_processes = 0;
    int     total_processes  = 0;
    std::string health_status = "HEALTHY";
};

// IOCTL Command Suite
constexpr unsigned long VDEV_IOCTL_START      = 0x8001;
constexpr unsigned long VDEV_IOCTL_STOP      = 0x8002;
constexpr unsigned long VDEV_IOCTL_RESET     = 0x8003;
constexpr unsigned long VDEV_IOCTL_GET_STATS = 0x8004;
constexpr unsigned long VDEV_IOCTL_SET_RATE  = 0x8005;

struct DeviceStats {
    uint64_t total_bytes_written = 0;
    uint64_t total_reads         = 0;
    uint64_t total_ioctls        = 0;
    uint64_t total_queries       = 0;
};
```

3. VERIFICATION & TEST RESULTS

3.1 Automated CTest Suite Output

Both test suites compiled and executed successfully with a 100% pass rate:

```
Test project /mnt/f/Projects/Wipro-Embedded-Training/Wipro Individual
Project/vdevpulse/build
  Start 1: device_test
1/2 Test #1: device_test ..... Passed    0.01 sec
  Start 2: telemetry_test
2/2 Test #2: telemetry_test ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 2
Total Test time (real) =  0.10 sec
```

3.2 Live CLI Execution Verification

Demonstrating status dashboard, JSON streaming, history buffer, and query protocol:

```
$ ./vdevpulse status
=====
          VDEVPULSE SYSTEM TELEMETRY DASHBOARD
=====
System Health      : HEALTHY
CPU Usage          : 0.58 %
RAM Memory Usage   : 539 MB / 7942 MB (6.8 %)
Load Averages      : 0.35 (1m), 0.10 (5m), 0.03 (15m)
Active Processes   : 1 running / 181 total
System Uptime      : 880 seconds
-----

$ ./vdevpulse status --json
{
  "health_status": "HEALTHY",
  "cpu_usage_pct": 0.58,
  "memory_used_mb": 539,
  "memory_total_mb": 7942,
  "memory_usage_pct": 6.79,
  "load_1m": 0.35,
  "load_5m": 0.10,
  "load_15m": 0.03,
  "running_processes": 1,
  "total_processes": 181,
  "uptime_seconds": 880
}

$ ./vdevpulse history
=====
          VDEVPULSE HISTORICAL TELEMETRY BUFFER
=====
Sample  CPU (%)    RAM (MB/%)    Load (1m)    Health
-----
1       0.6       539M/6%       0.35          HEALTHY
2       0.6       535M/6%       0.35          HEALTHY
3       0.6       535M/6%       0.35          HEALTHY
-----
```

```
$ ./vdevpulse query GET_CPU
CPU_PCT=0.58

$ ./vdevpulse query GET_HEALTH
HEALTH=HEALTHY

$ ./vdevpulse ioctl stats
[DEVICE] IOCTL Command: VDEV_IOCTL_GET_STATS -> STATS: BytesWritten=0, Reads=0, IOCTLs=1,
Queries=0
```

Conclusion: VDevPulse successfully integrates multi-source Linux kernel procfs telemetry, POSIX virtual character device stream I/O, automated threshold health alerts, historical buffering, and structured JSON export in a lightweight C++17 daemon for embedded systems.